

UNIVERSIDAD NACIONAL DEL ALTIPLANO
FACULTAD DE INGENIERÍA DE MECÁNICA ELÉCTRICA,
INGENIERÍA ELECTRÓNICA E INGENIERÍA DE SISTEMAS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



Interfaz de la gestión de disco

CURSO.
SISTEMAS OPERATIVOS

DOCENTE.
RUELAS ACERO DONIA ALIZANDRA

ALUMNO.
ABAD POMA MAQUERA

CODIGO.
241177

QUINTO SEMESTRE
2026 – I

PUNO – PERÚ

Introducción

Uno de los temas que más se discuten en la teoría de Sistemas Operativos, pero que pocas veces se logra visualizar de forma práctica, es la manera en que el sistema operativo decide dónde guardar cada archivo dentro del disco. Cuando se guarda un documento, el sistema tiene que responder de inmediato: ¿guardo los bloques juntos o los reparto por donde haya espacio? Esa decisión, que parece simple, tiene un impacto enorme en el rendimiento del sistema a largo plazo, ya que afecta tanto la velocidad de lectura como el aprovechamiento real del espacio disponible.

Este informe documenta el desarrollo de SimuKernel, un simulador interactivo desarrollado por el autor para poder visualizar de primera mano cómo funcionan los tres métodos más importantes de asignación de espacio en disco: Contigua, Enlazada e Indexada. En lugar de quedarse únicamente con la teoría del libro, el objetivo fue construir una herramienta que permitiera comparar en tiempo real qué pasa con el disco cuando se usa cada método, cuántas operaciones genera y qué tan fragmentado queda el almacenamiento.

Objetivo General

Desarrollar y documentar una interfaz gráfica interactiva orientada a simular, de forma visual y paso a paso, el comportamiento algorítmico de los métodos de asignación de espacio en disco (Contigua con variantes First Fit y Best Fit, Enlazada e Indexada), permitiendo la evaluación técnica del rendimiento de Entrada/Salida, el análisis de fragmentación y la comprensión del directorio de metadatos del sistema de archivos, todo ello mediante métricas calculadas en tiempo real.

Marco Teórico

Para construir el simulador con rigor, el trabajo partió de los tres modelos de asignación que aparecen en la bibliografía clásica de Sistemas Operativos. A continuación se explica cómo funciona cada uno y qué implicaciones tiene su uso en la práctica.

Asignación Contigua

Este método exige que el archivo se almacene en una secuencia ininterrumpida de bloques físicos dentro del disco. Su principal ventaja radica en el altísimo rendimiento durante la lectura secuencial; al estar los bloques adyacentes, el cabezal del disco realiza una única operación de búsqueda y lee la información de forma continua. Sin embargo, este enfoque adolece gravemente de un fenómeno conocido como fragmentación externa. A medida que se crean y eliminan archivos de diferentes tamaños, el disco comienza a llenarse de pequeños huecos libres dispersos. Con el tiempo, aunque la suma total de estos huecos sea suficiente para almacenar un archivo nuevo, el sistema operativo denegará la operación porque no existe un único hueco continuo del tamaño requerido.

El usuario define la entrada con tres campos por archivo: nombre, bloque de inicio y longitud en bloques. Para gestionar automáticamente la búsqueda del espacio libre, el simulador incorpora dos estrategias adicionales: el método *First Fit* (Primer Ajuste), que recorre el mapa del disco desde el bloque cero y asigna el primer hueco libre que tenga capacidad suficiente, logrando una asignación muy rápida; y el método *Best Fit* (Mejor Ajuste), que escanea la totalidad del disco buscando el hueco libre que más se ajuste al tamaño del archivo, minimizando el espacio sobrante, aunque a costa de un mayor tiempo de procesamiento.

Asignación Enlazada

A diferencia de la asignación contigua, el método enlazado elimina la restricción de adyacencia. El archivo puede estar disperso y fragmentado en cualquier bloque libre del disco. Para mantener la cohesión del archivo, cada bloque de datos almacena, además de la información del usuario, un pequeño puntero (configurable en el simulador, con un valor predeterminado de 4 bytes) que indica la dirección lógica del siguiente bloque de la secuencia. El último bloque de la cadena lleva un puntero especial con valor -1, indicando el fin del archivo. De esta forma, el directorio del sistema solo necesita conocer el bloque de inicio y el bloque final.

Si bien esta solución erradica por completo la fragmentación externa, introduce dos problemas significativos. El primero es la sobrecarga de espacio, ya que un porcentaje de cada bloque se sacrifica para guardar el puntero; el simulador calcula automáticamente los “datos

útiles por bloque” como `tamaño_bloque -- tamaño_puntero`. El segundo es la ineficiencia en el acceso directo: si se necesita leer el bloque cincuenta de un archivo, el sistema operativo está obligado a leer secuencialmente los cuarenta y nueve bloques anteriores para seguir el rastro de punteros.

Asignación Indexada

La asignación indexada nace como una solución híbrida para resolver el problema del acceso aleatorio de las listas enlazadas, sin caer en la fragmentación externa de la asignación contigua. Este método dedica un bloque de disco entero, conocido como bloque índice, exclusivamente a almacenar las direcciones de todos los bloques de datos del archivo. La capacidad máxima de este bloque índice está determinada por la fórmula: $\text{máx_índices} = \text{tamaño_bloque} / \text{tamaño_puntero}$. Para un bloque de 512 bytes y un puntero de 4 bytes, el bloque índice puede referenciar hasta 128 bloques de datos.

Cuando se crea un archivo, el bloque índice se escribe primero y en él se anotan todas las direcciones. Si el sistema necesita el bloque de datos número cincuenta, simplemente lee el índice y salta directamente a la ubicación física, sin lecturas intermedias. Sin embargo, el simulador evidencia la principal desventaja: el espacio desperdiciado en el bloque índice se calcula como `tamaño_bloque -- (cantidad_datos * tamaño_puntero)`, y ese espacio es irrecuperable, incluso si el archivo es muy pequeño.

Arquitectura y Tecnología

A la hora de construir el simulador, se tomó una decisión importante desde el principio: no usar librerías ni frameworks externos. El objetivo fue que el proyecto fuera liviano y que cualquier persona pudiera abrirlo en un navegador sin necesidad de instalar nada. Por eso todo está desarrollado con JavaScript puro (ES6), HTML5 y CSS3, lo que también exigió pensar bien cómo organizar la arquitectura del código.

El proyecto se dividió en tres módulos bien diferenciados. El primero, `algoritmos.js`, es el corazón matemático: no toca la pantalla para nada, solo procesa los datos que el usuario ingresa y genera una lista de “instantáneas” del disco, una por cada bloque que se escribe.

Esta arquitectura fue la que permitió lograr la animación paso a paso que hace que la simulación sea tan visual. El segundo módulo, `discoRenderer.js`, se encarga de dibujar: toma cada instantánea y pinta la grilla, los colores de cada archivo, los números de los punteros y las animaciones. El tercero, `mainDisco.js`, actúa como el director de orquesta: gestiona los botones de la pantalla, lee lo que el usuario escribió en la tabla o cargó desde un CSV, y controla el reproductor con sus cinco niveles de velocidad.

Para mostrar la grilla del disco se utilizó CSS Grid, lo que permitió representar el disco como una cuadrícula numerada muy fácil de leer. Además, al módulo de disco se le asignó un color violeta/morado propio para que se distinga visualmente del resto de componentes del sistema.

Resultados y Métricas

Lo destacable del simulador al probarlo es que no se queda solo en lo visual. Mientras los bloques se van pintando en la pantalla, el sistema calcula en tiempo real cuánto espacio se está usando, qué tan fragmentado está quedando el disco y cuántas veces tuvo que escribir físicamente en él. A continuación se explica qué significa cada una de esas métricas y cómo se obtiene.

Porcentaje de Uso del Disco

Esta métrica refleja la densidad de almacenamiento del disco en un momento dado. Incluye tanto bloques de datos como bloques de índice, ofreciendo una imagen fiel del espacio total consumido. En sistemas reales, cuando este indicador supera el ochenta por ciento, el rendimiento general comienza a degradarse notablemente. Su fórmula es la siguiente:

$$\% \text{ Uso del Disco} = \frac{\text{Bloques Ocupados}}{\text{Total de Bloques}} \times 100$$

Fragmentación Externa

La fragmentación externa es calculada por el simulador escaneando el mapa completo del disco y contando la cantidad de grupos o “islas” de bloques libres no contiguos. Cada grupo

separado de espacio libre por bloques ocupados se contabiliza como una unidad de fragmentación. Este indicador es especialmente crítico bajo la asignación contigua, y es irrelevante bajo los métodos enlazado e indexado, donde cualquier bloque libre puede ser utilizado independientemente de su posición. Su fórmula de cálculo es:

$$\text{Fragmentación Externa} = \text{Nº de huecos libres separados en el disco}$$

Operaciones de Entrada/Salida (E/S)

Las operaciones de E/S son acumuladas por el simulador paso a paso, diferenciando lecturas (R) de escrituras (W). Este es el indicador de mayor impacto en el rendimiento físico del hardware. El costo de escritura varía drásticamente según el método de asignación seleccionado:

Para la asignación **Contigua**, el costo es el mínimo posible, equivalente a la cantidad exacta de bloques del archivo:

$$E/S_{\text{Contigua}} = N \text{ escrituras}$$

Para la asignación **Enlazada**, el costo en escrituras es igual al de la contigua, pero se añade una penalización en espacio útil por bloque que el simulador calcula como:

$$\text{Datos útiles por bloque} = \text{Tamaño de Bloque} - \text{Tamaño de Puntero}$$

Para la asignación **Indexada**, el costo sube obligatoriamente en una operación adicional por el bloque índice. Adicionalmente, el simulador calcula el espacio desperdiciado en el bloque índice y la capacidad máxima de punteros que puede alojar:

$$E/S_{\text{Indexada}} = N + 1 \text{ escrituras}$$

$$\text{Capacidad máxima del índice} = \left\lfloor \frac{\text{Tamaño de Bloque}}{\text{Tamaño de Puntero}} \right\rfloor$$

$$\text{Espacio desperdiciado} = \text{Tamaño de Bloque} - (\text{Nº datos} \times \text{Tamaño de Puntero})$$

Adicionalmente, el simulador renderiza en tiempo real el Directorio del Sistema de Archivos, también conocido como la tabla de metadatos. La estructura de esta tabla cambia dinámicamente según el algoritmo seleccionado, como se detalla en la Tabla 1.

Cuadro 1

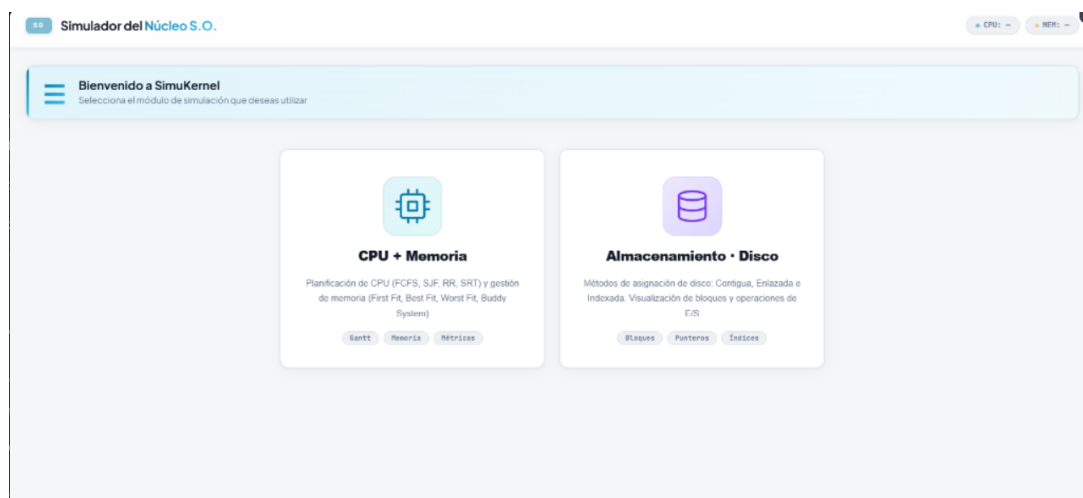
Estructura de la Tabla de Metadatos según el Método de Asignación

Método	Campos del Directorio	Descripción
Contigua	Archivo, Inicio, Longitud	Inicio y extensión de la región contigua
Enlazada	Archivo, Inicio, Final	Primer y último bloque de la cadena
Indexada	Archivo, Bloque Índice	Puntero al bloque centralizador de índices

La visualización en vivo de esta tabla permite al estudiante comprender cómo el sistema operativo mantiene el registro lógico de todos los archivos sin escanear el disco completo. A continuación se presentan las capturas del simulador en funcionamiento.

Figura 1

Pantalla de inicio dando la bienvenida a SimuKernel



La Figura 1 muestra la pantalla de bienvenida del sistema. Desde ahí el usuario elige si quiere trabajar con la parte de CPU y Memoria o entrar directamente al módulo de disco que desarrollamos para esta práctica.

Figura 2

Panel de configuración: selección del método y ajuste de parámetros físicos del disco

01 Configuración de Disco
Define los archivos y selecciona el método de asignación

Método de Asignación

☒ **Contigua**
Bloques adyacentes Acceso directo **SECUENCIAL**

☐ **Enlazada**
Lista enlazada Sin fragmentación externa **LISTA ENLAZADA**

☐ **Indexada**
Bloque de índices Acceso directo **BLOQUE INDICE**

VARIANTE DE ASIGNACIÓN CONTIGUA

Manual (inicio exacto) First Fit (Primer ajuste) Best Fit (Mejor ajuste)

CANTIDAD DE BLOQUES
120

TAMAÑO DE BLOQUE (BYTES)
512

Recomendado: 512 para replicar ejercicios del PDF

TAMAÑO PUNTERO/DIRECCIÓN (BYTES)
4

En la Figura 2 vemos la sección de configuración. Lo interesante aquí es que el usuario no solo elige el método, sino que también puede afinar los detalles físicos: para la asignación Contigua puede seleccionar si quiere definir el bloque de inicio a mano o dejar que el simulador lo calcule con First Fit o Best Fit. Más abajo se configuran la cantidad de bloques del disco (recomendamos 120 para que coincida con los ejercicios del material del curso), el tamaño de cada bloque en bytes y el tamaño del puntero, este último es clave porque el simulador lo usa para calcular cuánto espacio real le queda a cada bloque para datos de usuario en la asignación enlazada.

Figura 3

Zona de carga de archivos: importación CSV o edición manual adaptativa

Archivos en Disco

ARCHIVO DE DATOS (.CSV / .TXT)

Arrastra tu archivo aquí o haz clic
archivo, inicio, longitud
Sin archivo seleccionado

ARCHIVOS (EDICIÓN MANUAL)

ARCHIVO	INICIO	LONGITUD	
Archivo	Inicio	Longitud	✕

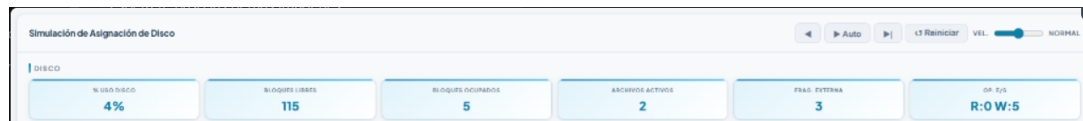
+ Agregar archivo

FORMATO (CONTIGUA - MANUAL)
archivo, inicio, longitud
AA, 1, 3
BB, 14, 2

La Figura 3 muestra las dos formas de ingresar los datos al simulador. La más rápida es arrastrar un archivo .csv o .txt directamente a la zona de carga; el simulador lo lee automáticamente y llena la tabla. La otra opción es escribirlo a mano fila por fila, y lo que nos pareció muy práctico es que las columnas de la tabla cambian solas según el método que hayas elegido, por lo que no hay que adivinar qué datos ingresar.

Figura 4

Panel de simulación mostrando métricas calculadas en tiempo real



En la Figura 4 se puede ver el panel de control durante la ejecución. Arriba están los botones del reproductor (retroceder, reproducir automáticamente, avanzar paso a paso y reiniciar) y el slider de velocidad, que tiene cinco niveles desde muy lento hasta máximo. Debajo aparece el panel de métricas que se actualiza en tiempo real conforme avanza cada paso de la simulación, mostrando desde el porcentaje de uso del disco hasta las operaciones de escritura acumuladas.

A continuación, se presentan los resultados de las tres simulaciones realizadas con los datos de prueba del material teórico del curso.

Para generar la simulación del método contiguo, se utilizaron los siguientes datos de entrada:

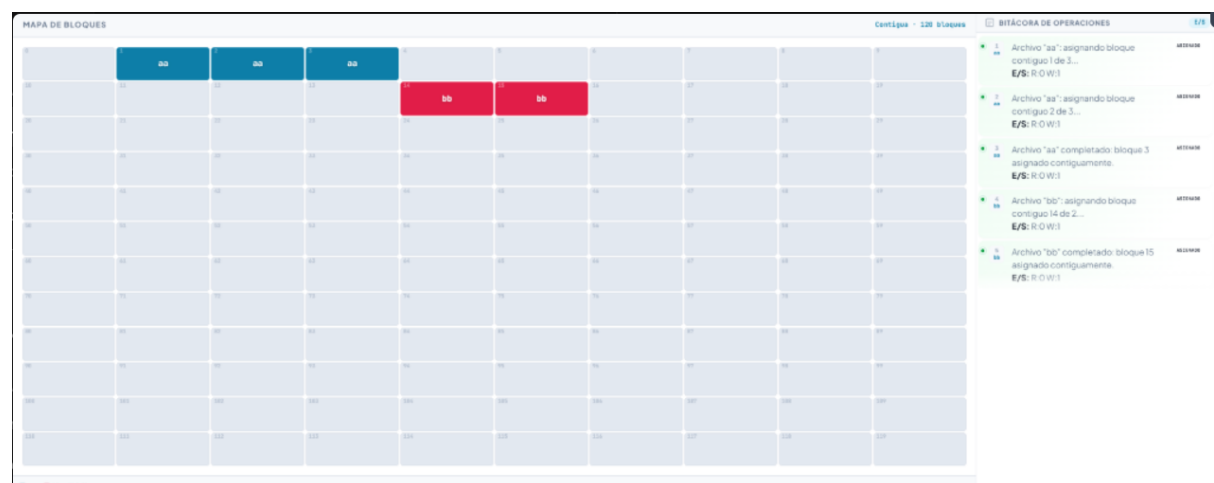
Cuadro 2

Datos de Entrada para la Simulación de Asignación Contigua

Archivo	Inicio	Longitud (bloques)
AA	1	3
BB	14	2

Figura 5

Mapa de bloques para el método contiguo: asignación secuencial de AA y BB



La Figura 5 muestra el resultado de la asignación contigua. Se nota claramente que AA quedó en los bloques 1, 2 y 3 (en azul) y BB en los bloques 14 y 15 (en rojo), con un hueco de 10 bloques vacíos en el medio. La bitácora de la derecha registró las 5 escrituras individuales que el simulador fue ejecutando una por una. Lo interesante es que aunque visualmente haya espacio, si intentáramos guardar un archivo de 11 bloques el sistema lo rechazaría porque no hay 11 bloques contiguos disponibles, y ese es exactamente el problema de la fragmentación externa que tanto discutimos en la teoría.

Para la prueba de la asignación enlazada, los datos ingresados fueron los siguientes:

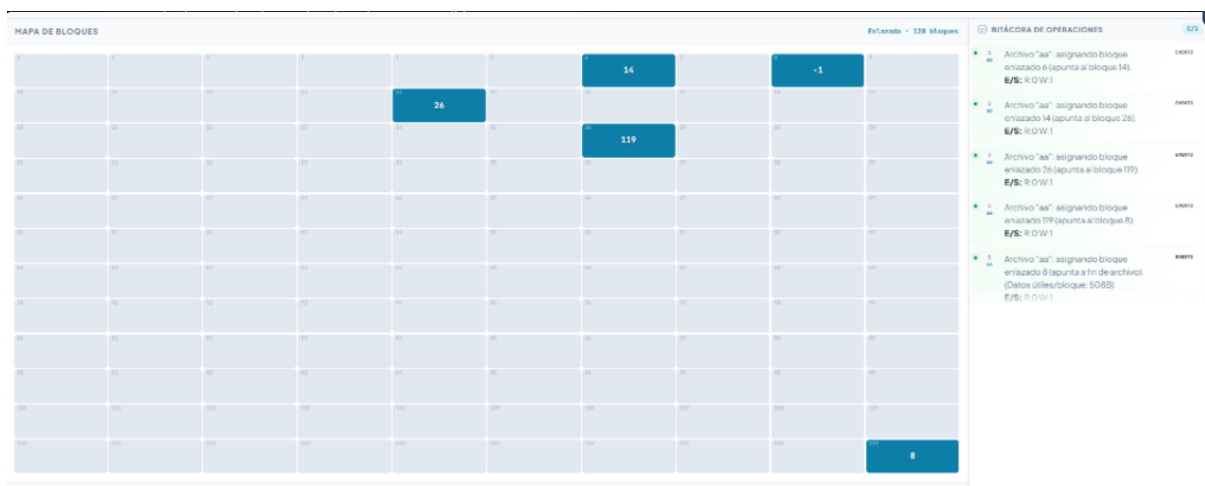
Cuadro 3

Datos de Entrada para la Simulación de Asignación Enlazada

Archivo	Secuencia de Bloques
AA	6, 14, 26, 119, 8

Figura 6

Mapa de bloques enlazado: archivo AA disperso con punteros visibles en cada celda



La Figura 6 es quizás la más llamativa visualmente. Se puede ver el archivo AA completamente disperso por el disco, saltando del bloque 6 al 14, luego al 26, después al 119 y finalmente al 8. Dentro de cada celda aparece el número del bloque siguiente, que es el puntero. El último bloque, el 8, muestra el valor -1 que indica que ahí termina el archivo. El simulador también nos mostró un dato concreto: con bloques de 512 B y punteros de 4 B, cada bloque solo puede guardar 508 B de datos reales, los otros 4 B se los come el puntero.

Finalmente, para la demostración de la asignación indexada, la configuración suministrada fue:

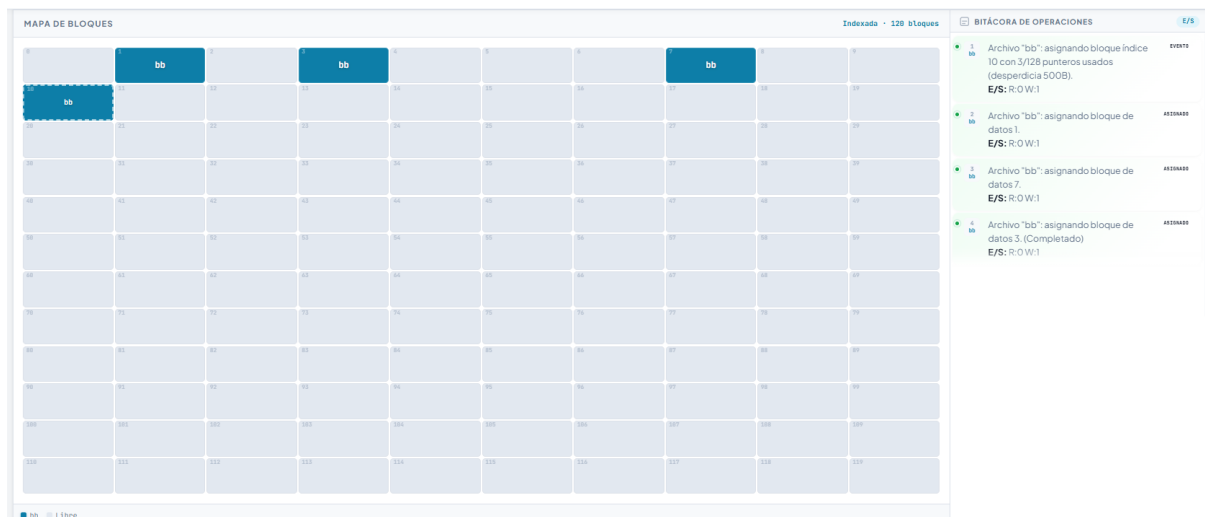
Cuadro 4

Datos de Entrada para la Simulación de Asignación Indexada

Archivo	Bloque Índice	Bloques de Datos
AA	10	1, 7, 3

Figura 7

Mapa de bloques indexado: bloque índice 10 referenciando los datos de AA



La Figura 7 muestra la asignación indexada. El bloque 10 aparece diferenciado visualmente como el bloque índice y desde él se apunta a los bloques de datos 1, 7 y 3. Lo que más llama la atención al revisar la bitácora es el primer mensaje que generó el simulador: el bloque índice solo usó 3 de los 128 punteros que podría tener, desperdiciando 500 de sus 512 bytes. Este dato, que aparece en pantalla, evidencia de manera muy gráfica el costo estructural que tiene este método cuando los archivos son pequeños.

Conclusiones

Desarrollar este simulador resultó ser una de las prácticas más enriquecedoras del curso, porque obligó a entender los algoritmos a fondo, no solo a memorizarlos. Al observar el disco animarse bloque por bloque en la pantalla, quedó muy claro algo que la teoría menciona pero que cuesta dimensionar: ningún método es perfecto. La asignación contigua es la más rápida y simple, pero basta con simular dos archivos para ver cómo empieza a aparecer ese hueco inutilizable en el medio del disco, validando empíricamente la fragmentación externa.

Con la asignación enlazada se comprendió que solucionar la fragmentación tiene un precio. Esos 4 bytes que se le restan a cada bloque para guardar el puntero parecen poca cosa, pero el simulador dejó en evidencia que en un disco real con millones de bloques ese “impuesto” se acumula de forma significativa. Además, la forma en que los bloques quedan dispersos por toda la grilla hace muy visual por qué el acceso directo al centro del archivo se vuelve lento.

La asignación indexada fue quizás la más sorprendente, sobre todo cuando el simulador reportó que el bloque índice del archivo AA desperdiciaba 500 de sus 512 bytes para guardar apenas tres punteros. Ese dato concreto, apareciendo en tiempo real en la bitácora mientras corría la simulación, resulta pedagógicamente más potente que cualquier párrafo de un libro de texto.

En resumen, SimuKernel cumplió con creces el objetivo planteado: convertir conceptos que suelen quedarse en lo abstracto en algo que se puede ver, controlar y medir. El hecho de que todo haya sido construido con tecnologías web puras demuestra además que no se necesitan herramientas complejas para desarrollar algo con valor didáctico real.

Referencias

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10ma ed.). Wiley.
- Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9na ed.). Pearson.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4ta ed.). Pearson.

Anexos

Anexo A: Repositorio del Proyecto

El código fuente completo del simulador SimuKernel se encuentra disponible de forma pública en el siguiente repositorio de GitHub:

<https://github.com/Abad-cyber/Sistema-del-nucleo-S.O..git>

El repositorio incluye todos los módulos del proyecto: los algoritmos de simulación, el motor de renderizado visual, los estilos CSS y la interfaz HTML principal, listos para ejecutarse localmente con cualquier servidor web estático.

Anexo B: Manual de Usuario (Resumen)

El uso del simulador de disco sigue tres pasos básicos:

1. **Seleccionar el módulo:** Desde la pantalla de inicio, hacer clic en la tarjeta *Almacenamiento · Disco*.
2. **Configurar los parámetros:** En la sección de parámetros, elegir el método de asignación (Contigua, Enlazada o Indexada), la variante si corresponde (Manual, First Fit o Best Fit), la cantidad de bloques del disco y el tamaño de bloque en bytes.
3. **Ingresar los archivos:** Escribir los datos directamente en la tabla de edición o cargar un archivo .csv o .txt arrastrándolo a la zona de carga. El formato varía por método:

- Contigua manual: nombre, inicio, longitud
- Enlazada: nombre, bloque1, bloque2, ...
- Indexada: nombre, bloque_indice, dato1, dato2, ...

4. **Ejecutar la simulación:** Pulsar el botón *Simular Disco*. La animación avanzará bloque por bloque de forma automática. Se puede pausar, retroceder, avanzar manualmente o ajustar la velocidad con el control deslizante (cinco niveles: Muy lento a Máximo).